Optimize inferencing GPT-OSS from scratch

Nam Vu

Khai Tong

Project Overview

Goal: Implement high-performance inference for gpt-oss models (20B & 120B parameters to achieve highest throughput (tokens/sec)

- Naive 's TPS: 1 (token/sec)

- Baseline: C language, run on CPU, based on llama2.c https://github.com/n30nd/Optimize_Inference_GPT-OSS, greedy decoding
- Tokenizer: Compatible with OpenAI o200k_harmony
- Models: Pre-exported binary weights from Hugging Face checkpoints
  - https://huggingface.co/openai/gpt-oss-20b https://huggingface.co/openai/gpt-oss-120b
- Parameter sizes: 20B, 120B
  - Interactive (chat)
  - Non-interactive:
    - Single prompt (generate)
    - Multiple prompts for final evaluation
      - Offline inference with fixed input prompts.
      - Limit accuracy score: Meteor (> 0.3) and Bertscore (> 0.9)
- Resources: 1 node, 8× AMD MI250 GPUs (64GB VRAM, HIP for GPU),
- Write all GPU kernels from scratch — no GPU libraries

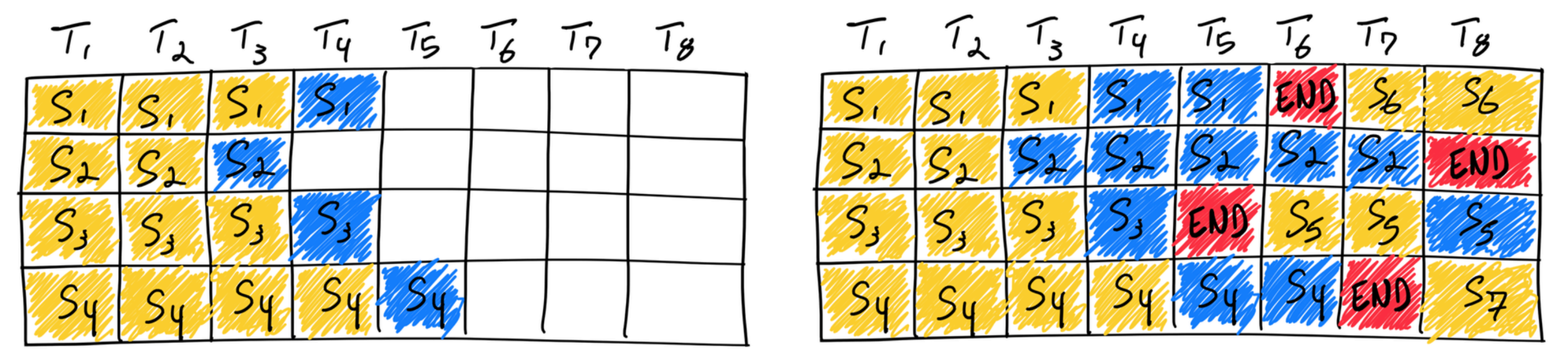
20B

- Port layer's CPU code to kernel on GPU
  - kernel: code run on GPU, with multiple thread parallel
- Quantize: Weights BF16, Runstate FP32
  - 1 model can fit on 1 GPU with ~40GB weights / 64GB VRAM
- Parallelism: Data Parallel = 8
  - 1 thread (CPU) manage 1 GPU
  - Each thread get request in atomic queue

20B

Continuous batching:

- Batch proccesing with multi SLOTS
- Once completely decode 1 slots, move to next request in stead of wait all slots complelte



20B

Profilie: Choose what to speedup/optimize

$$\text{Overall Speedup} = \frac{\text{Old execution time}}{\text{New execution time}}$$

$$= \frac{1}{\left(1 - \text{Fraction}_{\text{enhanced}} \right) + \left(\frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} \right)}$$

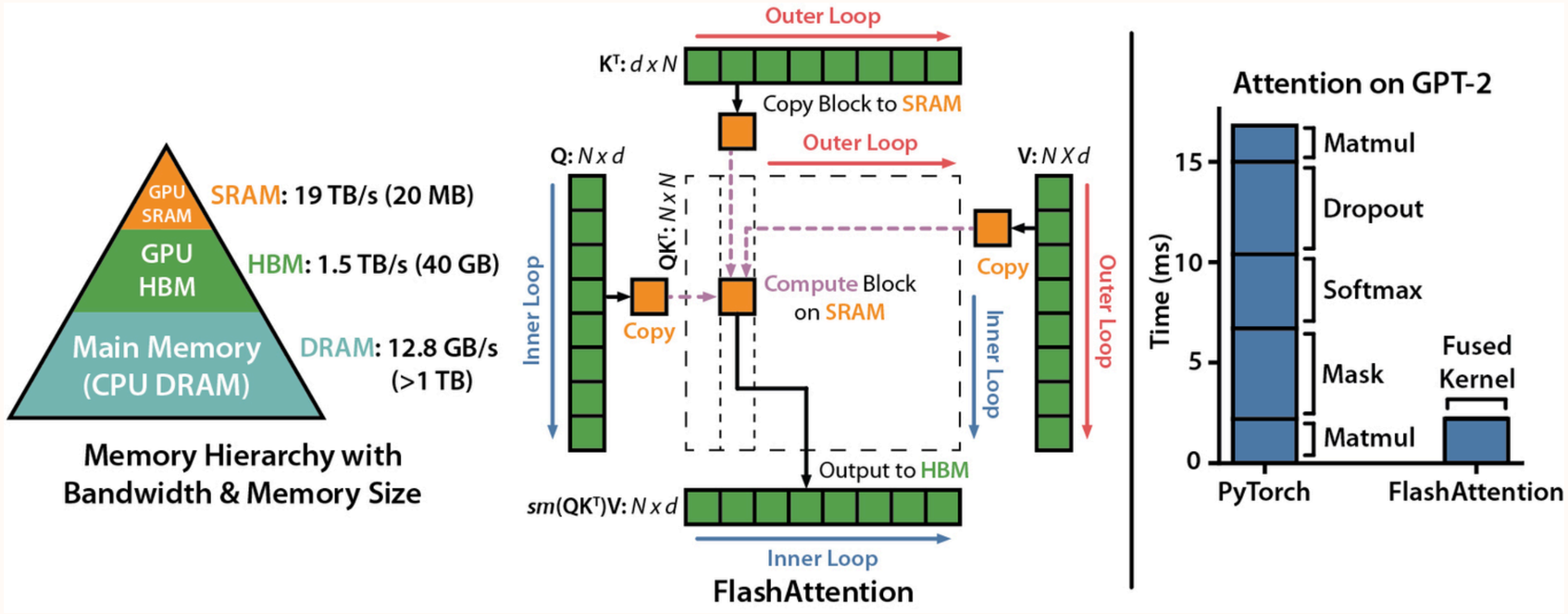
Amdahl law



20B

Flash Attention:

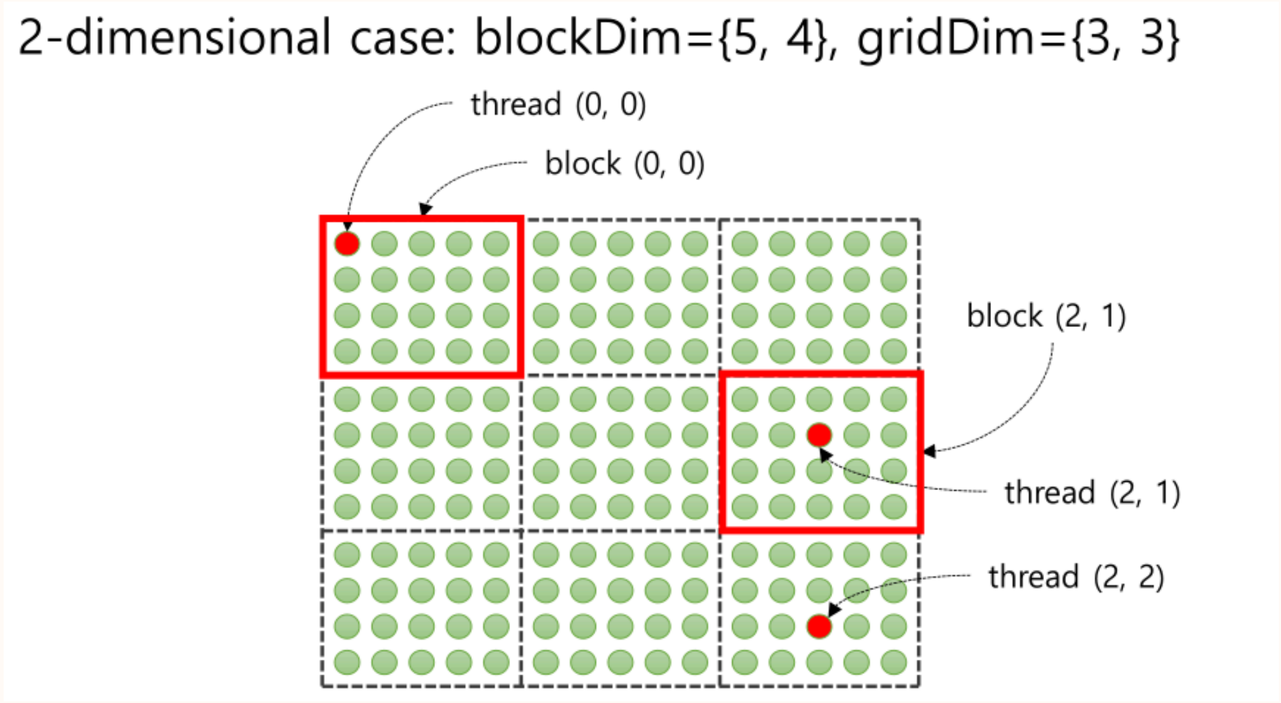
- Streaming softmax online, reduce write/read from global memory
- Compute tile Bq x Bk, Bv one time
  - For decode: Bq often=1
  - Flash decoding: Devide to multiple chungks of (BqxBk,Bv) parallel, then reduce



20B

- Optimize matmul
  - QKV, Router, logits, ...
  - MLP1, MLP2

Y[MxN] = W[MxK] @ X[KxN]



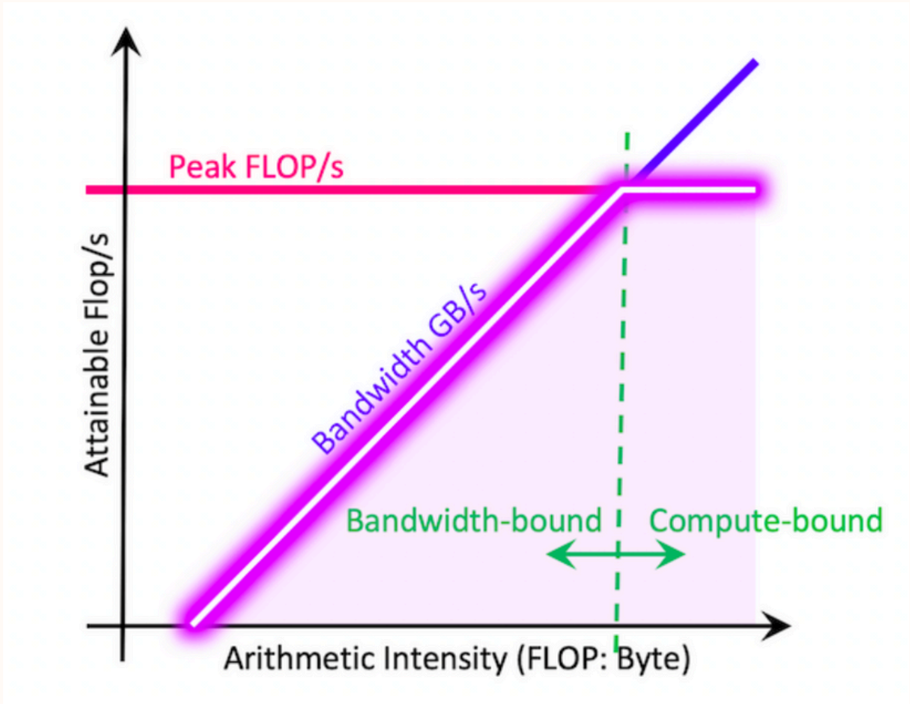
Choose GridDim has the same size with shape of output matrix

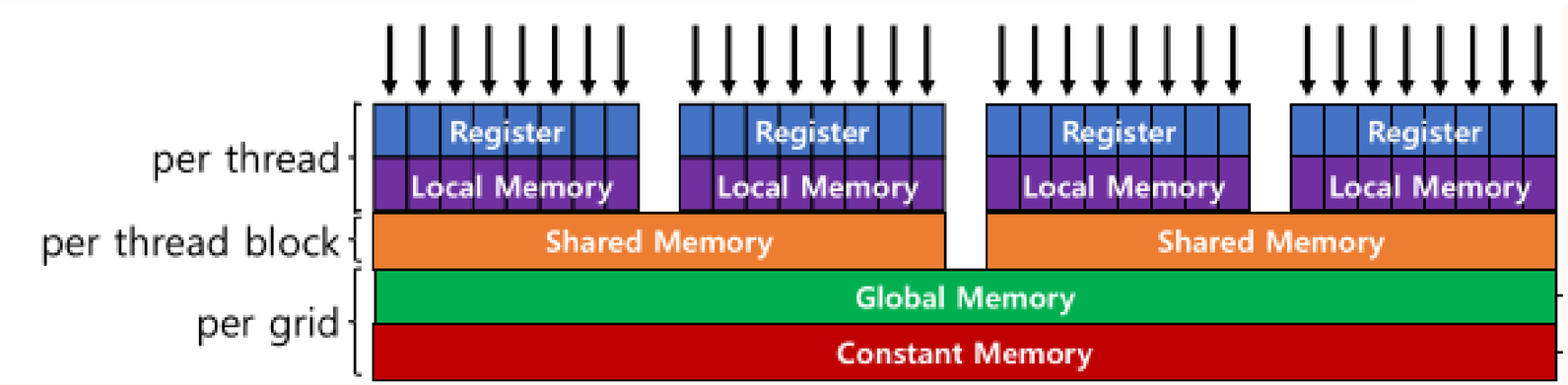


Architecture of SM, 1 GPU consist of multiple SMs
To simply: SM ~ 1 CPU multicore

20B

- Optimize kernel
  - Less memory bound, more compute bound
- Simple memory hierarchy: Global memory → shared mem (between thread in block) - register (thread use)
- Less memory bound: Optimize loading data
  - Global memory → Sharedmem
  - Sharedmem → register





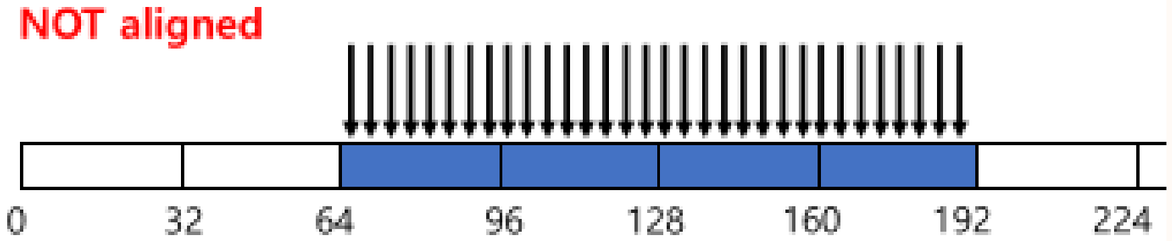
20B

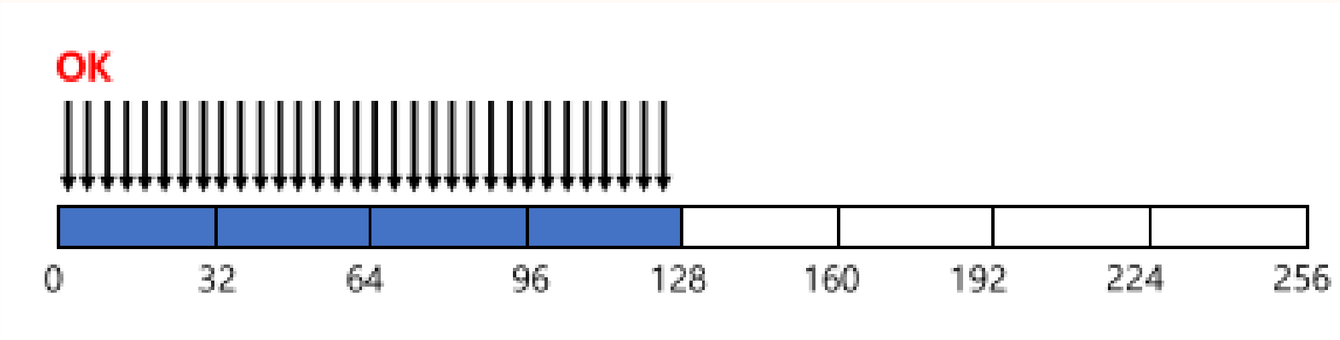
Global → SharedMem: Memory Coalescing

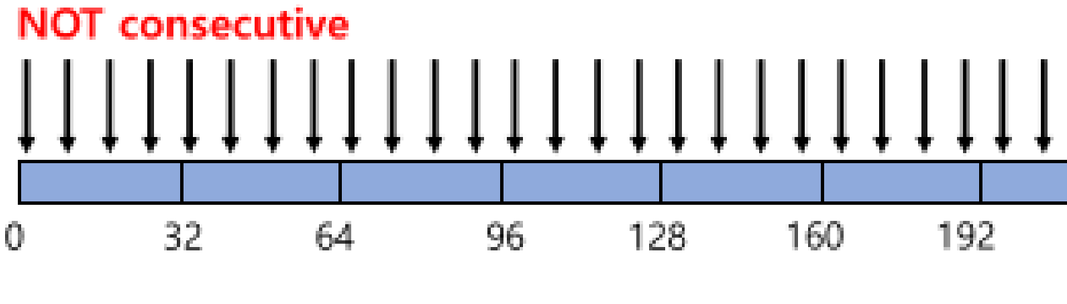
Merge multiple memory transactions into one transaction
- Hardware optimization in the memory subsystem

Two conditions for memory coalescing

- Consecutive accesses
  - Threads in the same warp have to access consecutive addresses
- Alignment
  - The memory access has to be 128-byte aligned

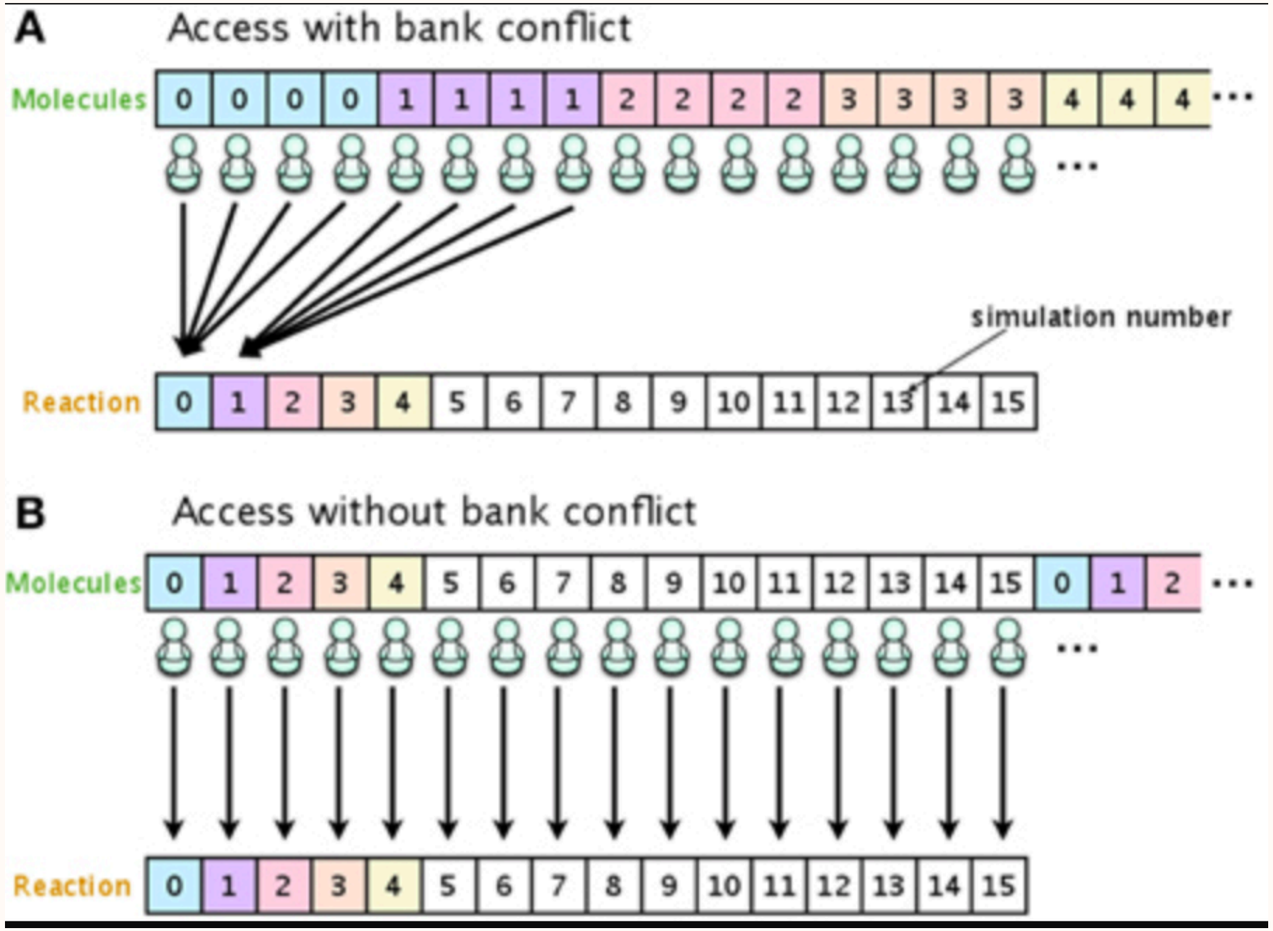






20B

- SharedMem → Register: Manage bank conflit



20B- Optimize matmul

MLP:

- **Group expert**
- W[M, K] @ X_pack[K, N]
  - M = outdim: intermediatedim x 2 (MLP1), hidden_dim (MLP2)
  - K = in_dim: hidden_dim (MLP1), intermediate_dim (MLP2)
  - N = #selections (padded per expert block)
- Use matrix core intrinsic __builtin_amdgcn_mfma_f32_32x32x8bf16_1k or(__builtin_amdgcn_mfma_f3216x16x16_1k)
- Block handles BM×BN tile; X_tile reused across wavefronts; wavefronts* ∝ BM, BN → tuning.
  - Intensity ∝ BM x BN / (BM + BN)
- Memory coalising:
  - Pack Weight to 32x8 tile when storage
  - Gather pack, transpose X before enter to GEMM
  - Vectorize load
- Double buffering
- Bank conflict: Tried padding, swizzle, dual-thread load on the same bank with split gate-up handling, single-thread bank load with sharing to the paired thread.
- Fuse kernel

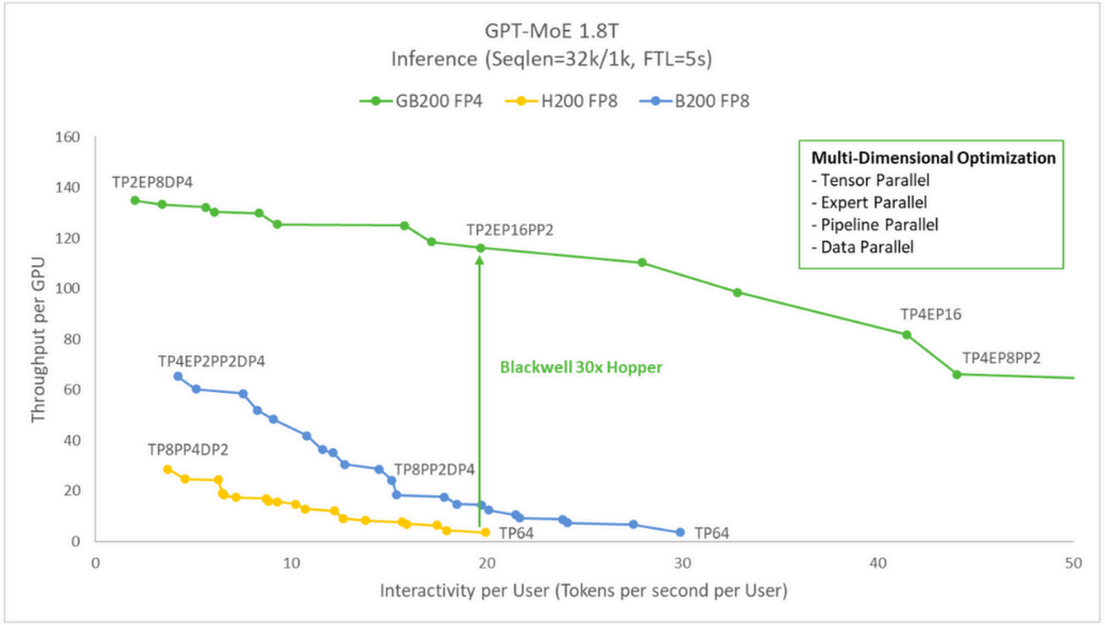
The BM/BN tile tuning idea is applied similarly to other GEMMs.

*Wavefront/warps is the smallest execution unit, consisting of 64/32 threads depending on HIP/CUDA

120B

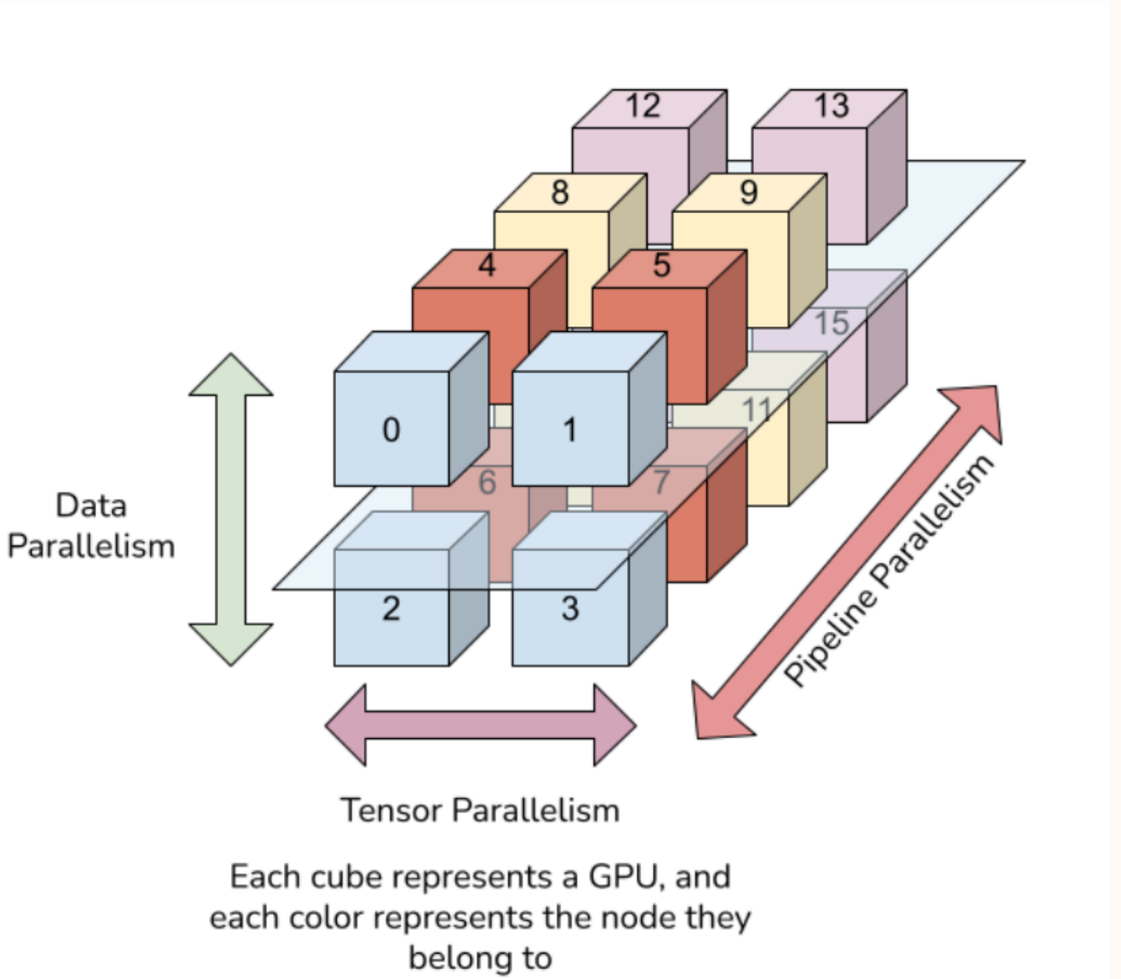
- Model don't fit on 1 GPU -> Scale to multiGPU with Parallelism

- Parallel tech:
  - Data Parallel: Dive data
  - Tensor Parallel: Divde compute-load, weight
  - Expert Parallel: Divide Expert
  - Pipeline Parallel: Divde layer



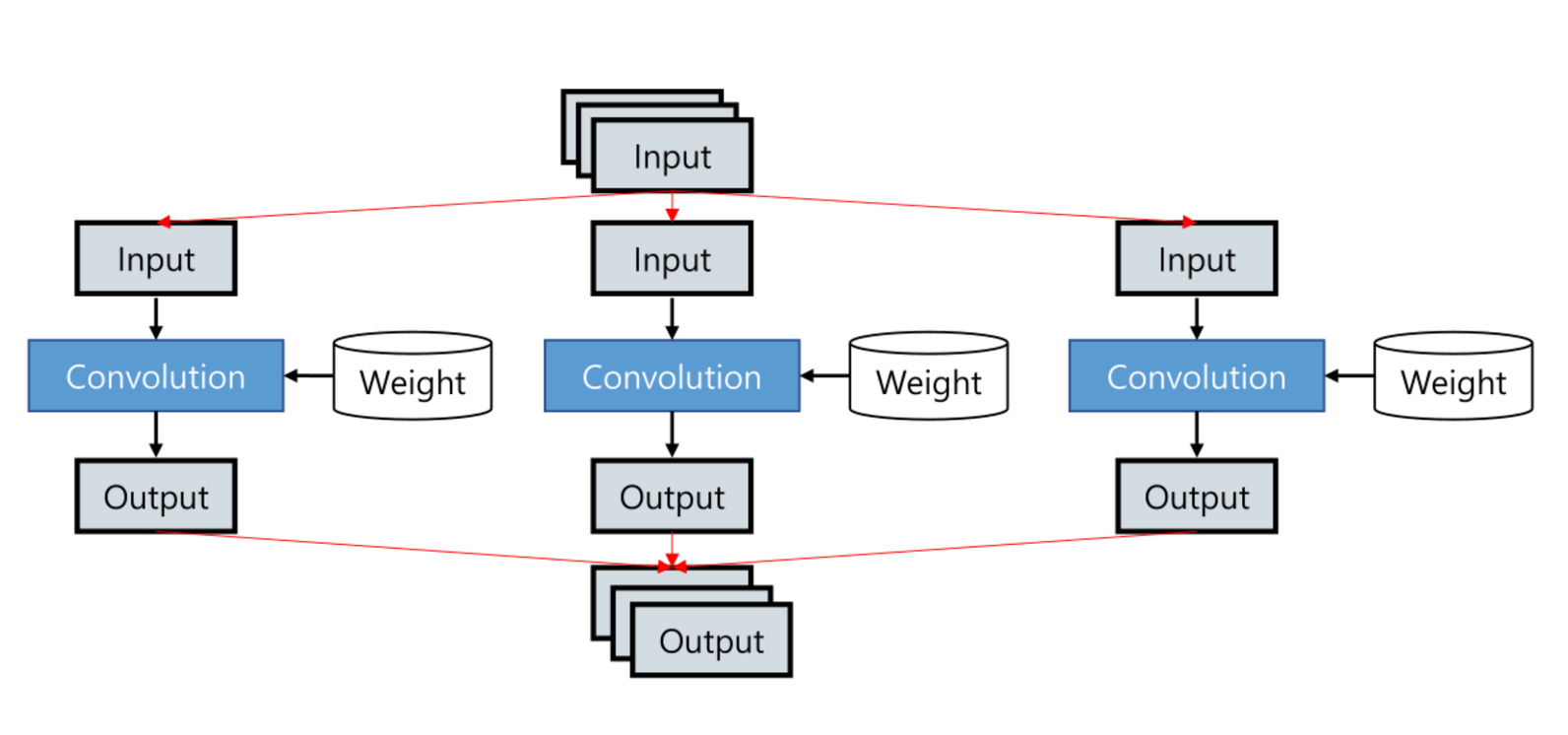
Peak throughput for different parallelism and chunking configurations for NVIDIA Blackwell, NVIDIA H100, and NVIDIA H200

https://developer.nvidia.com/blog/demystifying-ai-inference-deployments-for-trillion-parameter-large-language-models/



120B

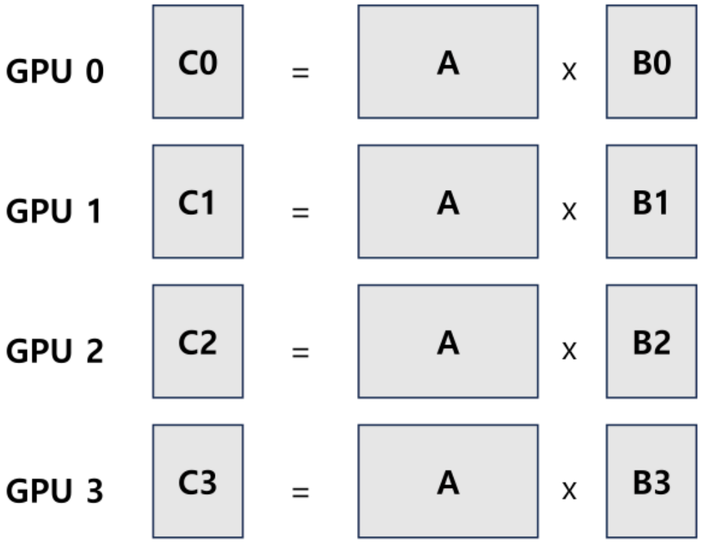
Data Parallel (DP): Clone model to GPU-group, divide data



120B

Tensor Parallelism (TP)

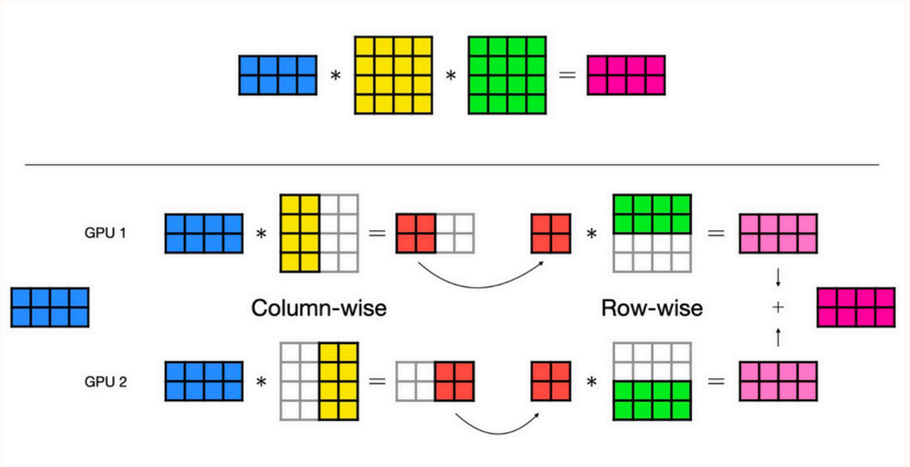
- Consider the matrix multiplication C = AB
  - Keep the same copies of A in all GPUs
  - Keep submatrices of B and C for each GPU



Exploit parallelism in tensor operations (e.g., matrix multiplication)
Steps in tensor parallel operations
1.Model parameters are partitioned across multiple GPUs
2.Each GPU applies the target operation using the partitioned parameters
- After this step, each GPU may have intermediate results

3.Additional synchronizations are applied to aggregate intermediate output activations
- To get the final results

Attention: divide by head
MLP: divide by dim
- MLP1: row
- MLP2: column



120B

Quantization & Precision

- Weights in BF16, except weight MLP1, MLP2 in MXFP4
- MXFP4: 4-bit symmetric level with ±6 limit, scale e8m0 (pow2) in 32-element blocks
  - Can fit model on 2 GPUs 64 GB VRAM, BATCH_SIZE 1024
  - Trade off: No support operation on AMD MI250 GPU, must dequantize when use.
- On-the-fly dequant in kernel: Use Look Up Table
-

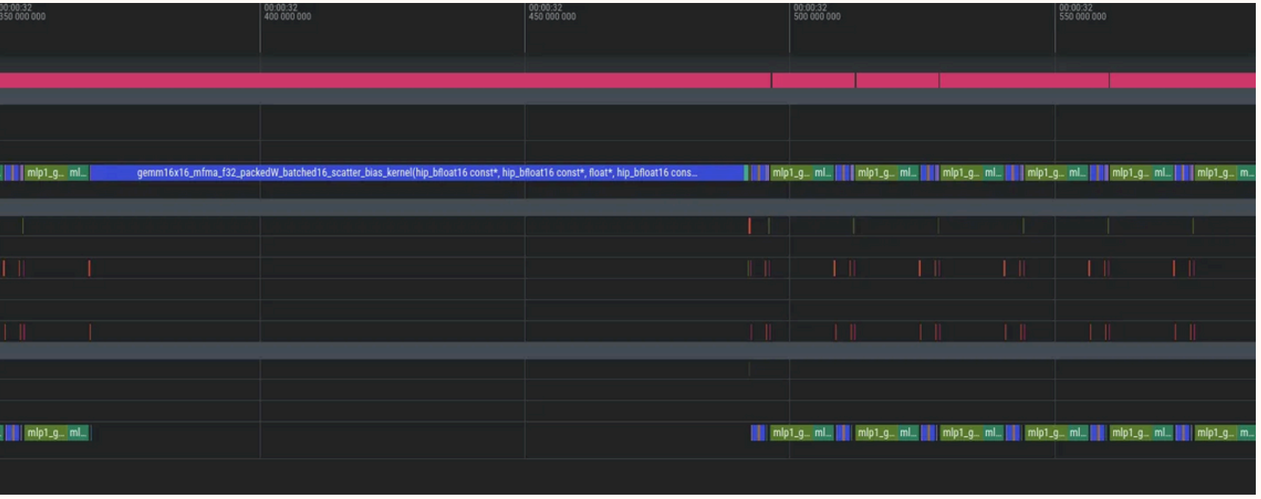
Format Name	Element Data Type	Element Bits (d)	Scaling Block Size (k)	Scale Data Type	Scale Bits (w)
MXFP8	FP8 (E5M2)	8	32	E8M0	8
	FP8 (E4M3)				
MXFP6	FP6 (E3M2)	6	32	E8M0	8
	FP6 (E2M3)				
MXFP4	FP4 (E2M1)	4	32	E8M0	8
MXINT8	INT8	8	32	E8M0	8

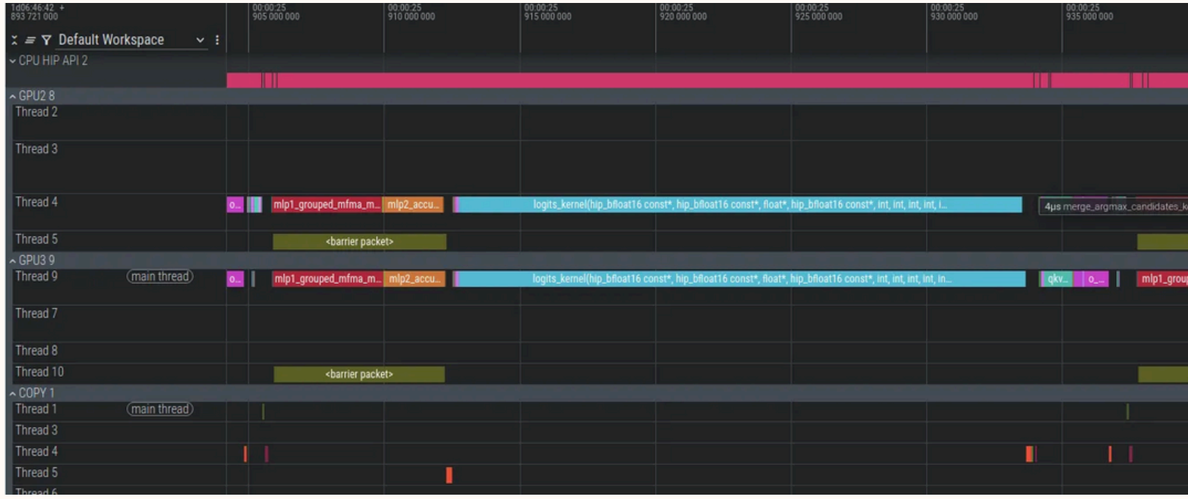
Table 1. Format names and parameters of concrete MX-compliant formats.

120B

Communication

- Double buffer ring reduce e_agg after MLP2
- Load divide Vocab / 2 for each GPU to calculate logits, fuse with argmax (due to greedy sampling) output the highest token_id of each GPU, call back to GPU0 to pick the real highest token_id





Logits kernel from one leader GPU to 2-GPU work and communication together

Merci pour votre écoute !

NOMADE